SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

sDry Run Evaluation

Code of Conduct:

I B Sandhya(192311292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandhya

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:

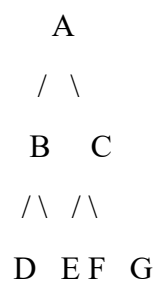
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

2. Write the BFS traversal order.
3. Show the queue contents after each iteration.

4. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

5. **Initial State :**

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

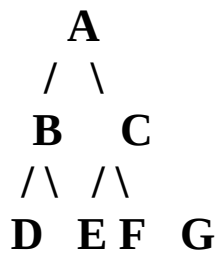
RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

ANSWERS

Question 1 – Breadth-First Search (BFS)

Given graph:



BFS visits nodes **level by level**, from left to right.

Dry Run

Iteration	Queue	Visited Node
1	`[B, C]`	A
2	`[C, D, E]`	B
3	`[D, E, F, G]`	C
4	`[E, F, G]`	D
5	`[F, G]`	E
6	`[G]`	F
7	`[]`	G

Step-by-Step Explanation

Initial:

Queue = [A]

Iteration 1: Remove A → visit A → add B, C

Queue = [B, C]

Iteration 2: Remove B → visit B → add D, E

Queue = [C, D, E]

Iteration 3: Remove C → visit C → add F, G

Queue = [D, E, F, G]

Iteration 4: Remove D → no children

Queue = [E, F, G]

Iteration 5: Remove E → no children

Queue = [F, G]

Iteration 6: Remove F → no children

Queue = [G]

Iteration 7: Remove G → no children

Queue = []

1. BFS Traversal Order

A → B → C → D → E → F → G

2. Queue Contents After Each Iteration

Initial : [A]

1. Visit A → [B, C]
2. Visit B → [C, D, E]
3. Visit C → [D, E, F, G]
4. Visit D → [E, F, G]
5. Visit E → [F, G]
6. Visit F → [G]
7. Visit G → []

Final Answer:
BFS = A, B, C, D, E, F, G

2.BFS Traversal Order

A → B → C → D → E → F → G

3. Show the queue contents after each iteration.

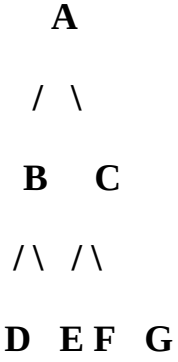
Queue Contents After Each Iteration

Iteration	Visited Node	Queue After Iteration
1	A	[B, C]
2	B	[C, D, E]
3	C	[D, E, F, G]
4	D	[E, F, G]
5	E	[F, G]
6	F	[G]
7	G	[]

Final queue: `[]` (empty)

4. Depth-First Search (DFS)

Using the same graph:



DFS explores **as deep as possible before backtracking**. Starting from **A**, and choosing the left child first:

Iteration	Stack	Visited Node
1	`[B, C]`	A
2	`[D, E, C]`	B
3	`[D, C]`	E
4	`[C]`	D
5	`[F, G]`	C
6	`[F]`	G
7	`[]`	F

DFS Traversal Order

A → B → E → D → C → G → F

Note: In iterative DFS, children are usually pushed onto the stack in reverse order so that the **left child is visited first**.

5. Breadth-First Search (BFS) – 8-Puzzle

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Note: The given table has only 5 rows, but the goal is **not reached within the first 5 expansions**. The shortest solution is reached at the **311th expansion** using the move-generation order **Up → Down → Left → Right**.

First 5 BFS Iterations

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	1 3 6 / 5 0 2 / 4 7 8	[Initial]	[1 0 6 / 5 3 2 / 4 7 8, 1 3 6 / 5 7 2 / 4 0 8, 1 3 6 / 0 5 2 / 4 7 8, 1 3 6 / 5 2 0 / 4 7 8]
2	1 0 6 / 5 3 2 / 4 7 8	[1 3 6 / 5 7 2 / 4 0 8, 1 3 6 / 0 5 2 / 4 7 8, 1 3 6 / 5 2 0 / 4 7 8]	[1 3 6 / 5 7 2 / 4 0 8, 1 3 6 / 0 5 2 / 4 7 8, 1 3 6 / 5 2 0 / 4 7 8, 0 1 6 / 5 3 2 / 4 7 8, 1 6 0 / 5 3 2 / 4 7 8]
3	1 3 6 / 5 7 2 / 4 0 8	[1 3 6 / 0 5 2 / 4 7 8, 1 3 6 / 5 2 0 / 4 7 8, 0 1 6 / 5 3 2 / 4 7 8, 1 6 0 / 5 3 2 / 4 7 8]	[1 3 6 / 0 5 2 / 4 7 8, 1 3 6 / 5 2 0 / 4 7 8, 0 1 6 / 5 3 2 / 4 7 8, 1 6 0 / 5 3 2 / 4 7 8, 1 3 6 / 5 7 2 / 0 4 8, 1 3 6 / 5 7 2 / 4 8 0]
4	1 3 6 / 0 5 2 / 4 7 8	[1 3 6 / 5 2 0 / 4 7 8, 0 1 6 / 5 3 2 / 4 7 8, 1 6 0 / 5 3 2 / 4 7 8, 1 3 6 / 5 7 2 / 0 4 8, 1 3 6 / 5 7 2 / 4 8 0]	[1 3 6 / 5 2 0 / 4 7 8, 0 1 6 / 5 3 2 / 4 7 8, 1 6 0 / 5 3 2 / 4 7 8, 1 3 6 / 5 7 2 / 0 4 8, 1 3 6 / 5 7 2 / 4 8 0, 0 3 6 / 1 5 2 / 4 7 8, 1 3 6 / 4 5 2 / 0 7 8]
5	1 3 6 / 5 2 0 / 4 7 8	[0 1 6 / 5 3 2 / 4 7 8, 1 6 0 / 5 3 2 / 4 7 8, 1 3 6 / 5 7 2 / 0 4 8, 1 3 6 / 5 7 2 / 4 8 0, 0 3 6 / 1 5 2 / 4 7 8, 1 3 6 / 4 5 2 / 0 7 8]	[0 1 6 / 5 3 2 / 4 7 8, 1 6 0 / 5 3 2 / 4 7 8, 1 3 6 / 5 7 2 / 0 4 8, 1 3 6 / 5 7 2 / 4 8 0, 0 3 6 / 1 5 2 / 4 7 8, 1 3 6 / 4 5 2 / 0 7 8, 1 3 0 / 5 2 6 / 4 7 8, 1 3 6 / 5 2 8 / 4 7 0]

1. Which node is expanded in each iteration?

For the **first five iterations**:

1. **Iteration 1:** Initial state
2. **Iteration 2:** 1 0 6 / 5 3 2 / 4 7 8
3. **Iteration 3:** 1 3 6 / 5 7 2 / 4 0 8
4. **Iteration 4:** 1 3 6 / 0 5 2 / 4 7 8
5. **Iteration 5:** 1 3 6 / 5 2 0 / 4 7 8

The goal is eventually expanded at **iteration 311**.

2. How many states are generated in total?

Using BFS with **Up → Down → Left → Right** move ordering and avoiding duplicate states:

500 unique states are generated/discovered before the goal is reached.

The goal is the **311th state expanded**.

3. Complete Solution Path

The shortest solution contains **8 moves**:

Initial

1 3 6

5 0 2

4 7 8

↓ Right

1 3 6

5 2 0

4 7 8

↓ Up

1 3 0

5 2 6

4 7 8

↓ Left

1 0 3

5 2 6

4 7 8

↓ Down

1 2 3

5 0 6

4 7 8

↓ Left

1 2 3

0 5 6

4 7 8

↓ Down

1 2 3

4 5 6

0 7 8

↓ Right

1 2 3

4 5 6

7 0 8

↓ Right

1 2 3

4 5 6

7 8 0

Solution Moves

Right → Up → Left → Down → Left → Down → Right → Right

4. Why does BFS always find the shortest solution?

BFS explores the state space **level by level**.

For the 8-puzzle:

- Level 0 → initial state
- Level 1 → states reachable in 1 move
- Level 2 → states reachable in 2 moves
- Level 3 → states reachable in 3 moves
- and so on.

Therefore, BFS reaches the goal first through the **minimum number of moves**.

Since every puzzle move has the same cost (**1 move**), BFS is guaranteed to find the **shortest solution path**.

Hence, BFS is complete and optimal for an unweighted 8-puzzle.